

UART(tx)

Universal Asynchronous Receiver / Transmitter

RTL Design, Testbench Development & Functional
Verification Report

Name/ Mahmoud Shawky

S26-B8-FPGA-G8

Objective

Design a configurable UART Transmitter.

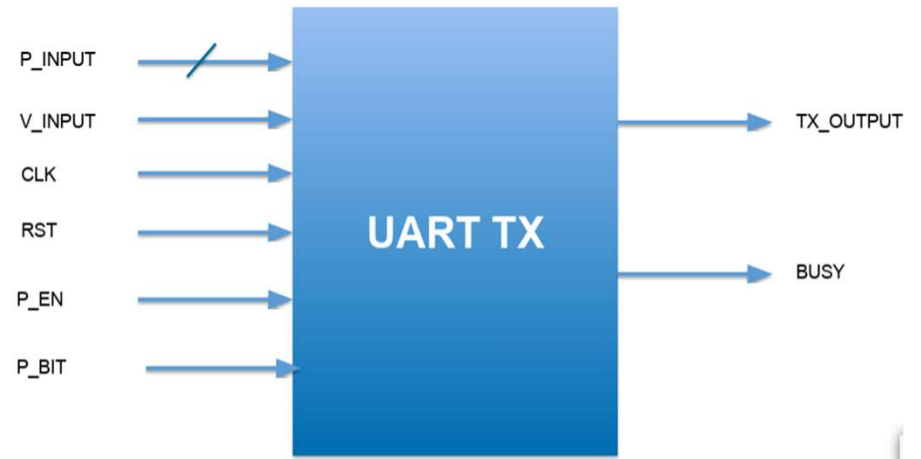
Transmit 8-bit parallel data serially.

Support optional parity bit.

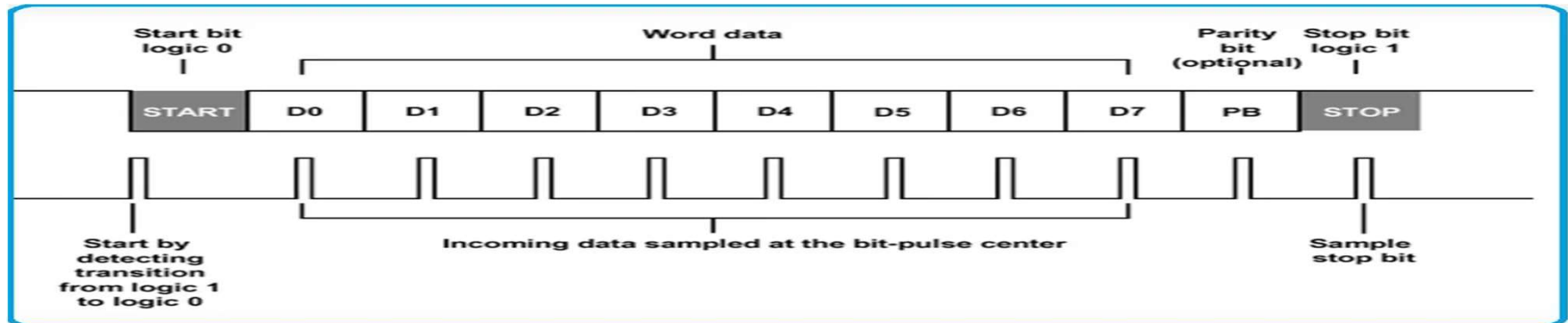
Support Even and Odd parity.

Generate Start and Stop bits.

Control transmission using an FSM.



System and specification



#	Frame Composition	Condition
1	Start bit + 8 data bits (LSB first) + Even parity bit + Stop bit	PAR_EN = 1, PAR_TYP = 0
2	Start bit + 8 data bits (LSB first) + Odd parity bit + Stop bit	PAR_EN = 1, PAR_TYP = 1
3	Start bit + 8 data bits (LSB first) + Stop bit (no parity)	PAR_EN = 0

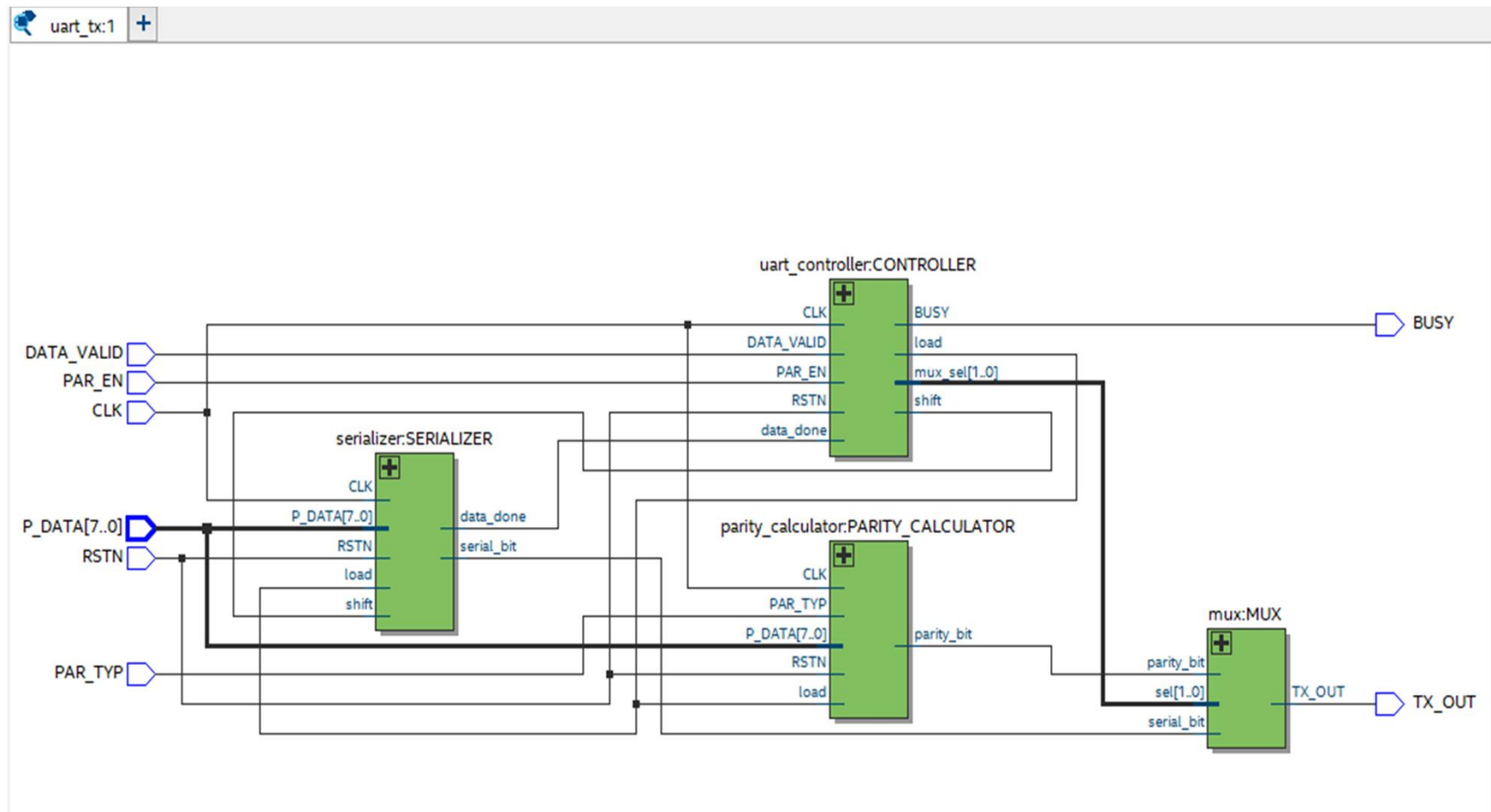
Transmitter (UART_TX):

- o Accepts 8-bit parallel input.
- o Generates start and stop bits.
- o Outputs serial data stream.
- o Provides status signals

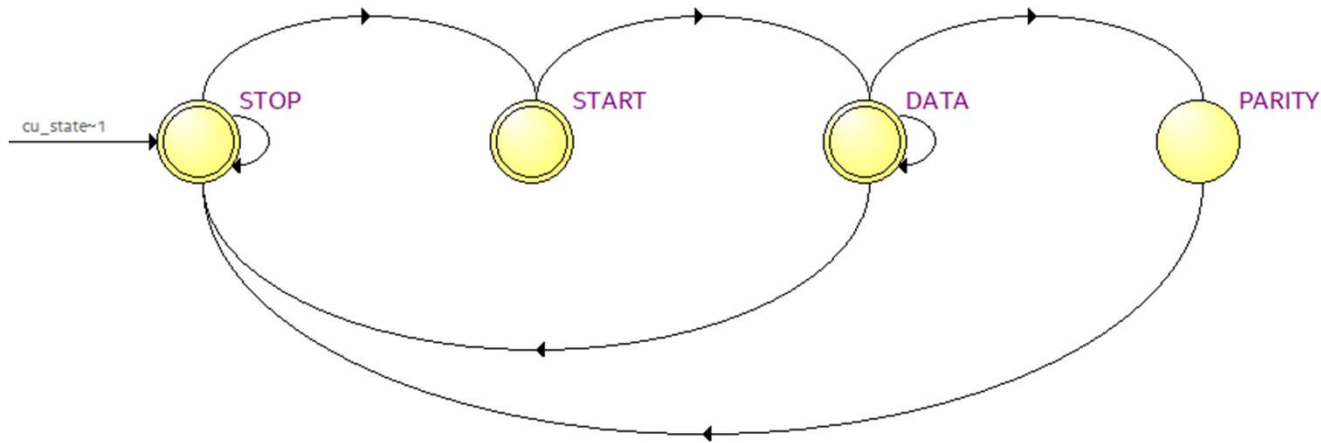
I/O Port Definition (Top-Level Modules)

Port	Direction	Width	Description
CLK	Input	1 bit	Clock signal
RSTN	Input	1 bit	Active-low reset
PAR_EN	Input	1 bit	Enables parity bit
PAR_TYP	Input	1 bit	Selects even or odd parity
P_DATA	Input	n bits	Parallel input data
DATA_VALID	Input	1 bit	Indicates valid input data
TX_OUT	Output	1 bit	Serial UART output
BUSY	Output	1 bit	Indicates transmission status

System Architecture



FSM(Controller)



Current State	Condition	Next State	Description
STOP	DATA_VALID = 1	START	Start new transmission
STOP	DATA_VALID = 0	STOP	Wait for new data
START	—	DATA	Start bit finished
DATA	data_done = 0	DATA	Continue sending data
DATA	data_done = 1 & PAR_EN = 1	PARITY	Send parity bit
DATA	data_done = 1 & PAR_EN = 0	STOP	Skip parity
PARITY	—	STOP	Parity bit finished

FSM(Controller)

I ensure that if anything happens in the design such as a 'reset' we start from the 'stop' state. This is because, in the 'stop' state, `busy` is 0, which means we can send a new frame. (beginning)

```
STOP: begin
    mux_sel = 2'b01;
    BUSY=1'b0;
    if(DATA_VALID)
        load = 1'b1;
    end
endcase
end

always @(posedge CLK or negedge RSTN) begin
    if(!RSTN)
        cu_state <= STOP;
    else
        cu_state<=next_state;
    end
end
```



The transition

```
always @(*) begin
    next_state = cu_state;

    case(cu_state)
        START: next_state = DATA;

        DATA: begin
            if(data_done)
                next_state = PAR_EN ? PARITY : STOP;
            end

        PARITY: next_state = STOP;

        STOP: begin
            if(DATA_VALID)
                next_state = START;
            end
    end
end
```

```
always @(*) begin
    load=1'b0;
    shift=1'b0;
    mux_sel=2'b01;
    BUSY=1'b1;

    case(cu_state)

        START: mux_sel=2'b00;

        DATA: begin
            mux_sel = 2'b10;
            shift = 1'b1;
        end

        PARITY: mux_sel = 2'b11;

        STOP: begin
            mux_sel = 2'b01;
            BUSY=1'b0;
            if(DATA_VALID)
                load = 1'b1;
            end
        endcase
    end
end
```

serialization

```
output serial_bit,
output reg data_done
);

reg [n-1:0] shift_reg;
reg [$clog2(n)-1:0] cnt;

assign serial_bit=shift_reg[0];

always @(posedge CLK or negedge RSTN) begin
if(!RSTN) begin
    shift_reg<=0;
    cnt<=0;
    data_done<=1'b0;
end
else begin
    data_done<=1'b0;

    if(load) begin
        shift_reg<=P_DATA;
        cnt<=0;
    end
    else if(shift) begin
        if(cnt == n-1) begin
            cnt <= 0;
        end
    end
    else begin
        shift_reg <= shift_reg >> 1;
        cnt <= cnt + 1'b1;
    end

    if(cnt == n-2) data_done <= 1'b1;
end
```

$$2^n = \text{Log}_2(n)$$

The **Serializer** operates on each **positive edge of the clock (CLK)**. When load = 1, it loads the parallel data, then shifts the data out **serially (LSB first)** when shift = 1. After transmitting the last data bit, data_done is asserted to indicate that the data transmission is complete

Here, I ensure the first bit is empty so I can receive the first bit from the LSB of the parallel input.

I need to output the "data done" signal early here so that the control logic can set `busy = 0` coincident with the last bit of the stop state.

Testbench

Signal Declarations

```
`timescale 1ns/1ps

module tb_uart_tx();

    parameter N = 8;

    reg CLK;
    reg RSTN;
    reg PAR_EN;
    reg PAR_TYP;
    reg DATA_VALID;
    reg [N-1:0] P_DATA;

    wire TX_OUT;
    wire BUSY;

    uart_tx #(N) DUT (
        .CLK(CLK),
        .RSTN(RSTN),
        .PAR_EN(PAR_EN),
        .PAR_TYP(PAR_TYP),
        .DATA_VALID(DATA_VALID),
        .P_DATA(P_DATA),
        .TX_OUT(TX_OUT),
        .BUSY(BUSY)
    );

    always #5 CLK = ~CLK;

    always @(posedge CLK) begin
        if(BUSY || DATA_VALID) begin
```

Test Cases

```
            $display("\n--- [Test 1] Data = 8'b1010_1101 | Parity ON");

            @(posedge CLK);
            P_DATA = 8'b10101101;
            PAR_EN = 1'b1;
            PAR_TYP = 1'b0;
            DATA_VALID = 1'b1;

            @(posedge CLK);
            DATA_VALID = 1'b0;

            wait(BUSY == 1'b1);
            wait(BUSY == 1'b0);
            #30;

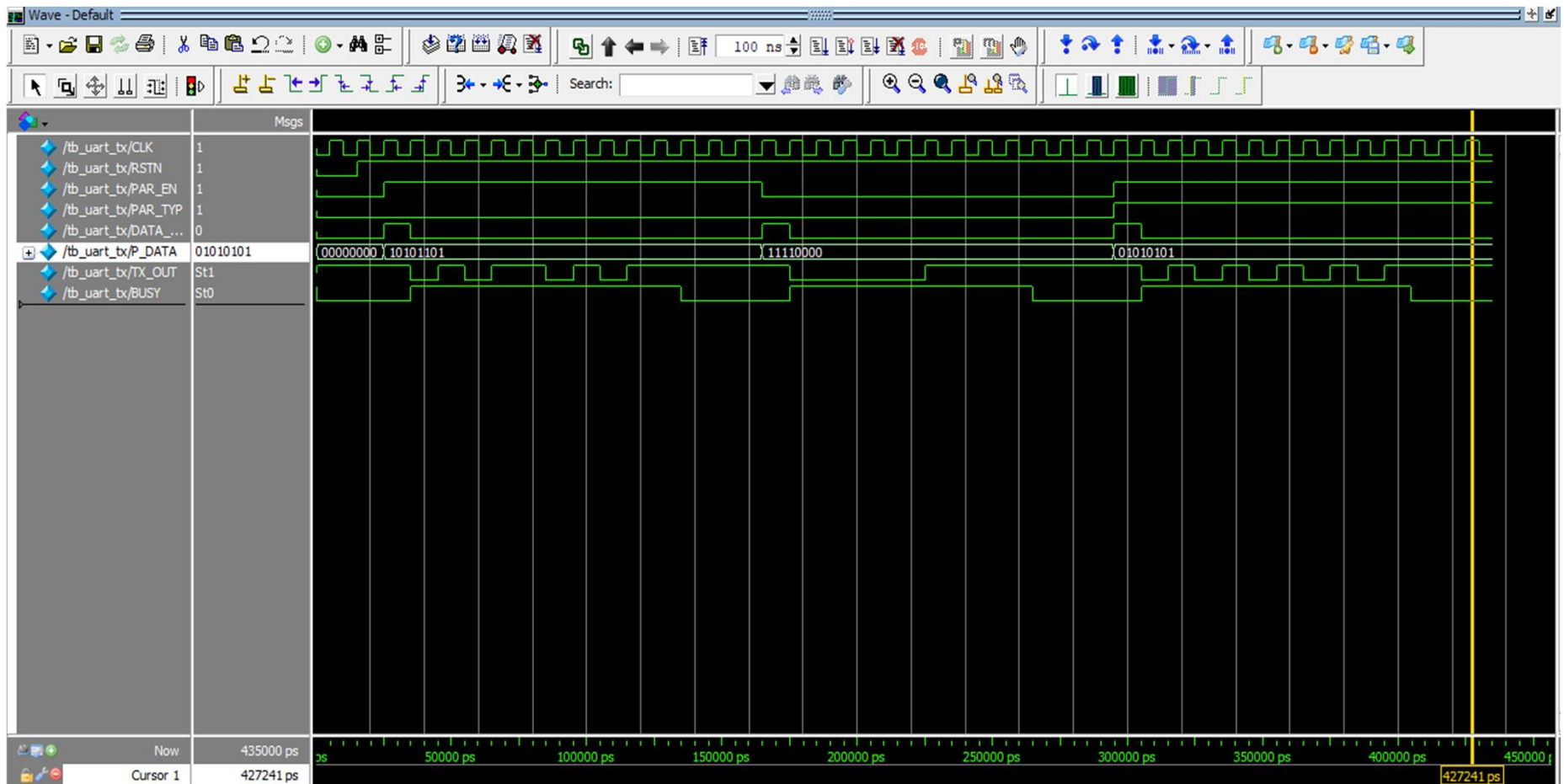
            $display("\n--- [Test 2] Data = 8'b1111_0000 | Parity OFF");

            @(posedge CLK);
            P_DATA = 8'b11110000;
            PAR_EN = 1'b0;
            DATA_VALID = 1'b1;

            @(posedge CLK);
            DATA_VALID = 1'b0;
            wait(BUSY == 1'b1);
            wait(BUSY == 1'b0);
            #30;

            $display("\n--- [Test 3] Data = 8'b0101_0101 | Parity ON");
            $display("\n=== Simulation Finished Successfully ===");
            $finish;
        end
    end
```

Wave form



Testbench(terminal)

```
--- [Test 1] Data = 8'b1010_1101 | Parity ON (Even) ---
Time=35000 ps | BUSY=0 | TX_OUT=1
Time=45000 ps | BUSY=1 | TX_OUT=0
Time=55000 ps | BUSY=1 | TX_OUT=1
Time=65000 ps | BUSY=1 | TX_OUT=0
Time=75000 ps | BUSY=1 | TX_OUT=1
Time=85000 ps | BUSY=1 | TX_OUT=1
Time=95000 ps | BUSY=1 | TX_OUT=0
Time=105000 ps | BUSY=1 | TX_OUT=1
Time=115000 ps | BUSY=1 | TX_OUT=0
Time=125000 ps | BUSY=1 | TX_OUT=1
Time=135000 ps | BUSY=1 | TX_OUT=1

--- [Test 2] Data = 8'b1111_0000 | Parity OFF ---
Time=175000 ps | BUSY=0 | TX_OUT=1
Time=185000 ps | BUSY=1 | TX_OUT=0
Time=195000 ps | BUSY=1 | TX_OUT=0
Time=205000 ps | BUSY=1 | TX_OUT=0
Time=215000 ps | BUSY=1 | TX_OUT=0
Time=225000 ps | BUSY=1 | TX_OUT=0
Time=235000 ps | BUSY=1 | TX_OUT=1
Time=245000 ps | BUSY=1 | TX_OUT=1
Time=255000 ps | BUSY=1 | TX_OUT=1
Time=265000 ps | BUSY=1 | TX_OUT=1

--- [Test 3] Data = 8'b0101_0101 | Parity ON (Odd) ---
Time=305000 ps | BUSY=0 | TX_OUT=1
Time=315000 ps | BUSY=1 | TX_OUT=0
Time=325000 ps | BUSY=1 | TX_OUT=1
Time=335000 ps | BUSY=1 | TX_OUT=0
Time=345000 ps | BUSY=1 | TX_OUT=1
Time=355000 ps | BUSY=1 | TX_OUT=0
Time=365000 ps | BUSY=1 | TX_OUT=1
Time=375000 ps | BUSY=1 | TX_OUT=0
Time=385000 ps | BUSY=1 | TX_OUT=1
Time=395000 ps | BUSY=1 | TX_OUT=0
Time=405000 ps | BUSY=1 | TX_OUT=1

=== Simulation Finished Successfully ===
** Note: $finish      : D:/mo/project_uart_tx_nti/tp.v(93)
```

Time analysis and daleys

The maximum frequency to be used is 68.02 MHz, as determined from the datasheet.

Fmax Summary				
	Fmax	Restricted Fmax	Clock Name	Note
1	68.02 MHz	68.02 MHz	clk	

$$T_{max} = 1/f_{max} = t_{clk} = 14.7\text{ns}$$

$$T_{clk} \geq t_{pd} + t_{cq} + t_{setup} \longrightarrow t_{setup} \text{ depends on } t_{clk}.$$

$$t_{hold} \leq t_{pd} + t_{cq} \longrightarrow t_{hold} \text{ not depends on } t_{clk}$$

Check

Test1

use clk less than Tclk = 10ns

Setup Violation

File View Netlist Constraints Reports Script Tools Window Help

Set Operating Conditions

☒ Slow 1100mV 85C Model
☐ Slow 1100mV 0C Model
☐ Fast 1100mV 85C Model
☐ Fast 1100mV 0C Model

Report

Summary (Setup)

	Clock	Slack	End Point TNS
1	clk	-2.488	-3.711

Tsetup It was affected.

Thold It was not affected.

File View Netlist Constraints Reports Script Tools Window Help

Set Operating Conditions

☒ Slow 1100mV 85C Model
☐ Slow 1100mV 0C Model
☐ Fast 1100mV 85C Model
☐ Fast 1100mV 0C Model

Summary (Hold)

	Clock	Slack	End Point TNS
1	clk	0.386	0.000

0.000

check

Test2

-Make clk gerter than Tclk = 20ns

File View Netlist Constraints Reports Script Tools Window Help Search altera.com

Set Operating Conditions

- ☒ Slow 1100mV 85C Model
- ☐ Slow 1100mV 0C Model
- ☐ Fast 1100mV 85C Model
- ☐ Fast 1100mV 0C Model

Summary (Setup)

	Clock	Slack	End Point TNS
1	clk	11.198	0.000

File View Netlist Constraints Reports Script Tools Window Help Search altera.com

Set Operating Conditions

- ☒ Slow 1100mV 85C Model
- ☐ Slow 1100mV 0C Model
- ☐ Fast 1100mV 85C Model
- ☐ Fast 1100mV 0C Model

Summary (Hold)

	Clock	Slack	End Point TNS
1	clk	0.407	0.000

Conclusion

- This project successfully demonstrated the design and implementation of a UART Transmitter using Verilog HDL. The system was structured into modular components including the UART Controller, Serializer, Parity Calculator, and MUX, ensuring a clear and organized hardware design.
- The controller manages the transmission sequence through four states: START, DATA, PARITY, and STOP. The design supports 8-bit parallel data, optional parity generation, and both Even and Odd parity configurations. The Serializer converts the parallel data into serial form using LSB-first transmission.
- Simulation results using ModelSim verified the correct operation of the transmitter through different data patterns and parity configurations. The generated waveforms confirmed the correct sequence of the UART frame: Start Bit → Data Bits → Parity Bit → Stop Bit. Quartus Prime was used for RTL Synthesis and Static Timing Analysis (STA) to verify the implemented hardware structure and timing performance.-